

ePortfolio Report

Front-end fundamentals, demonstrated.

Author	Sachita
Submission	Junior Full Stack Developer Portfolio
Live site	https://sachita-assignment.vercel.app/
Sections	Full Stack, HTML, CSS, Bootstrap, JavaScript, Evidence

01

Full Stack Development

Full stack development is just a way of saying you can put together a whole website yourself, not only one bit of it. There's the **front-end** side, which is everything you actually see when you open a webpage, and then the **back-end**, which is all the server stuff going on in the background. Saving data, handling requests, that kind of thing.

So when someone says they're a "full stack developer", what they really mean is they can switch between the two sides without losing the plot. They'll design the UI one minute and configure the database it connects to the next. Looking at job ads, you'll see a lot of them ask for a specific stack. The big one is **MERN** (that's MongoDB, Express, React and Node.js), and then there's **LAMP** (Linux, Apache, MySQL, PHP). My course covers MERN mostly, so that's the one I've gotten used to.

On the **front-end** side, you're working with HTML for structure, CSS for the look of things, and JavaScript whenever you need something to actually react to a user. Click a button and a menu drops down? That's JavaScript doing the work. Once apps get bigger though, plain JavaScript starts becoming a pain to maintain, which is why most companies reach for something like React or Angular instead.

The **back-end** is where it starts getting a little trickier. It deals with logins, sending and receiving data, saving info, anything that needs to remember things between page loads. You'll write APIs as well, which are just URLs your front-end can call to grab or send data. So say you fill out a form and hit submit, the back-end picks up that data, makes sure nothing's missing or off, and saves it into a database like MySQL or MongoDB.

Demo on the site

Interactive request flow: clicking **Send Request** animates a packet travelling from the Front-End to the Back-End and Database, with a live status label updating at each stage ("Sending request...", "Querying database...", "Response received").

02

HTML: The Skeleton

HTML is short for HyperText Markup Language, and it sits underneath pretty much every webpage on the internet. People sometimes call it a programming language but it really isn't one. You can't write loops or do maths with it. All it really does is let you mark up your content using tags. Want a heading on the page? `<h1>` does that. Need a paragraph? `<p>`. You get the idea.

One of the more useful things I took away from this module was the idea of **semantic HTML**. Older websites used to be full of `<div>` tags wrapping basically everything in sight, which worked fine but wasn't great. HTML5 sorted that out by adding proper tags like `<header>`, `<nav>`, `<main>`, `<article>` and `<footer>`. They're not just there to make your code look tidier either, they help the browser and search engines work out what each section of the page is meant to be.

Why does any of this matter though? Mostly comes down to two things. The first one is **accessibility**. People who use screen readers depend on those tags so their software can read the page out in an order that makes sense. The second is **SEO**, short for Search Engine Optimization. Google scans these tags to understand what a page is about, and that has a knock-on effect on where it ends up in search results.

HTML can do more than just hold text on a page too. You can put together forms for user input with `<input>` and `<form>`, drop in a video using `<video>`, or lay out information in tables. All these elements end up forming what we call the DOM, short for Document Object Model. Think of it as a tree of every element on your page, and that tree is what CSS and JavaScript hook into when they need to style something or change it.

Demo on the site

A real semantic mini-page ("Brewline Coffee Co.") built with `<header>`, `<nav>`, `<main>`, `<h1>`, `<p>` and `<footer>` tags. The rendered preview matches the code snippet shown below it.

03

CSS: The Aesthetic

CSS is short for Cascading Style Sheets. The way I think about it, if HTML is the skeleton of a page, then CSS is what puts the clothes on it. Without it, every site on the internet would basically be plain black text on a white background and a bunch of bright blue links everywhere, which would get tired pretty quick. CSS takes care of all the visual side of things. The colours, the fonts, the spacing, how big things are, where they sit on the page, all of it.

The whole point of CSS is to keep design separate from content. Your HTML stays focused on what the page actually says and how it's structured, and your CSS handles how that all appears on screen. That split turns out to be really handy in practice. If you ever want to give your website a completely new look, you can just swap the CSS file out and never touch the HTML.

Two tools I've been using a lot lately are **Flexbox** and **CSS Grid**. Flexbox is the one I reach for when I need to line things up in one direction, like a row of buttons in a nav bar. Grid works better for a full page layout where you need rows and columns to work together. To make sites responsive, we use **media queries**. These let the layout shift around depending on whether someone is on a phone, a tablet, or a laptop.

Something I've grown pretty fond of is **CSS variables** (also called custom properties). You set up a colour or a value once at the top of your file, and then you can use it anywhere without retyping it. If you decide later to change the theme, you only need to update that one variable and everything else follows along. CSS also handles hover effects and basic animations, which means you can add a bit of movement or feedback to your page without writing any JavaScript at all.

Demo on the site

A real pricing card component built with Flexbox for layout, CSS variables (`--card-bg`, `--card-accent`, `--card-radius`) for theming, and a hover effect that lifts the card with a soft shadow.

04

Bootstrap Framework

Bootstrap is a CSS framework that started out at Twitter, put together by a couple of developers working there. The thinking behind it is actually pretty straightforward. Instead of sitting there writing CSS from scratch every time you build a website, Bootstrap hands you a load of ready-made classes that you just chuck onto your elements. It speeds things up massively, especially when you're putting something together under time pressure.

What it's best known for is the **grid system**. The page gets split into 12 invisible columns, and you decide how many each element takes up. Say you want two boxes side by side at equal widths, you put `col-md-6` on each (because 6 plus 6 makes 12, which checks out). The nice bit is, it handles being responsive for you. On a phone, those two boxes will automatically stack one on top of the other, no extra CSS needed from your side.

Bootstrap also comes packed with **components** that are already built for you. Navbars, buttons, cards, modals (which are basically pop-ups), dropdown menus, the lot. You don't need to style any of them yourself. Just throw the right class onto the element and it already looks decent. They behave the same way across browsers too, which sounds like a small thing until you've spent half a day trying to figure out why a button looks wrong in Safari.

With Bootstrap 5 they introduced a whole heap of **utility classes**, for stuff like margins, padding, text alignment, colours and so on. Instead of writing a full CSS rule just to bump up some padding, you can stick something like `p-3` straight onto the element itself. Not everyone is a fan of doing things this way since your HTML can start looking a bit messy, but when you're prototyping it's hard to argue with how quick it makes things. Under the hood, Bootstrap is built using Sass, so if you want to dig in and tweak the default fonts or colours you can do that too.

Demo on the site

A working Bootstrap contact form using `row g-3, col-md-6` for the name and email fields side by side on desktop (stacking on mobile), `col-12` for subject and message, plus `form-control`, `form-select` and `btn` classes.

05

JavaScript: The Behavior

JavaScript is the thing that gives a website life. HTML lays out the structure, CSS dresses it up, but without JavaScript nothing on the page would do anything when you click or type. It's a proper programming language in its own right. You can write functions, run loops, store values in variables, all the usual stuff. And the nice bit is, every browser these days comes with a JavaScript engine built in, so your code just runs straight away.

The main thing JavaScript does on a webpage is **mess with the DOM**. The DOM is basically how the browser sees your HTML, laid out as a tree of objects you can talk to in code. With JavaScript, you can grab any element you want and change it. Maybe you're updating the text inside a paragraph, hiding a section that was visible before, switching a button to a different colour when someone clicks it, or creating brand new elements out of thin air. It also keeps an eye out for events, so the second a user clicks something or types into a field, your code is ready to react.

Honestly, the one concept that took me a while to properly get my head around was **asynchronous JavaScript**. Sometimes you need to pull data from a server (say, loading posts from an API), and obviously you don't want your webpage to freeze up while it waits for a response. JavaScript deals with this using Promises, but there's a much nicer way of writing it called `async/await` that's a lot less of a headache to read. For actually sending those requests off, we tend to use the Fetch API. That's the reason modern websites can pull in fresh content without making you sit through a full page reload every time.

One of the best things about JavaScript these days is that it's not just stuck inside the browser anymore. Thanks to **Node.js**, you can run JavaScript on a server too, which means the same language can power your front-end and your back-end. On top of that you've got frameworks like **React**, **Angular** and **Vue**, which make building big, complicated web apps way more manageable. I've only just started getting into React myself, and yeah it's a bit of a step away from plain JS, but I can already see why people lean on it for the bigger projects.

Demo on the site

A working to-do list built with plain JavaScript: add tasks via input + button, click a task to mark it complete (strikethrough), and per-item delete buttons. Pure DOM manipulation — no framework, no backend.

06

Evidence

The live ePortfolio website demonstrating each topic above is hosted on Vercel. All five sections are interactive on the live site — the link below is the primary piece of evidence for this submission.

Live website

<https://sachita-assignment.vercel.app/>

Covered on the live site

- Full Stack Development — interactive request flow animation.
- HTML — working semantic mini-page demo.
- CSS — pricing card component using Flexbox, CSS variables, hover lift.
- Bootstrap — responsive contact form with col-md-6 grid layout.
- JavaScript — working to-do list with add / complete / delete via DOM manipulation.